Tiny code (SQL) - Fine-Tuning Lightweight Language Models for BigQuery SQL Generation

End-to-End Pipeline using QLoRA, GKE, and Rigorous Validation

Master in Applied Artificial Intelligence

AAI 590 IN1 Capstone Project

Team 8

Sanjay Kumar, Santa Das Karmakar, Sethuraman Krishnasamy

April 2026

University of San Diego

Problem Statement

Goal: Building a reliable LLM that converts natural language questions into executable BigQuery SQL

- Natural language to SQL (Text-to-SQL) is challenging for non-technical users
- General-purpose LLMs often hallucinate columns, use wrong dialect, or produce invalid SQL
- Need for **BigQuery/GoogleSQL** compatibility (not just generic SQL)
- Goal: High-accuracy, schema-grounded SQL generation with validation

Key Objectives

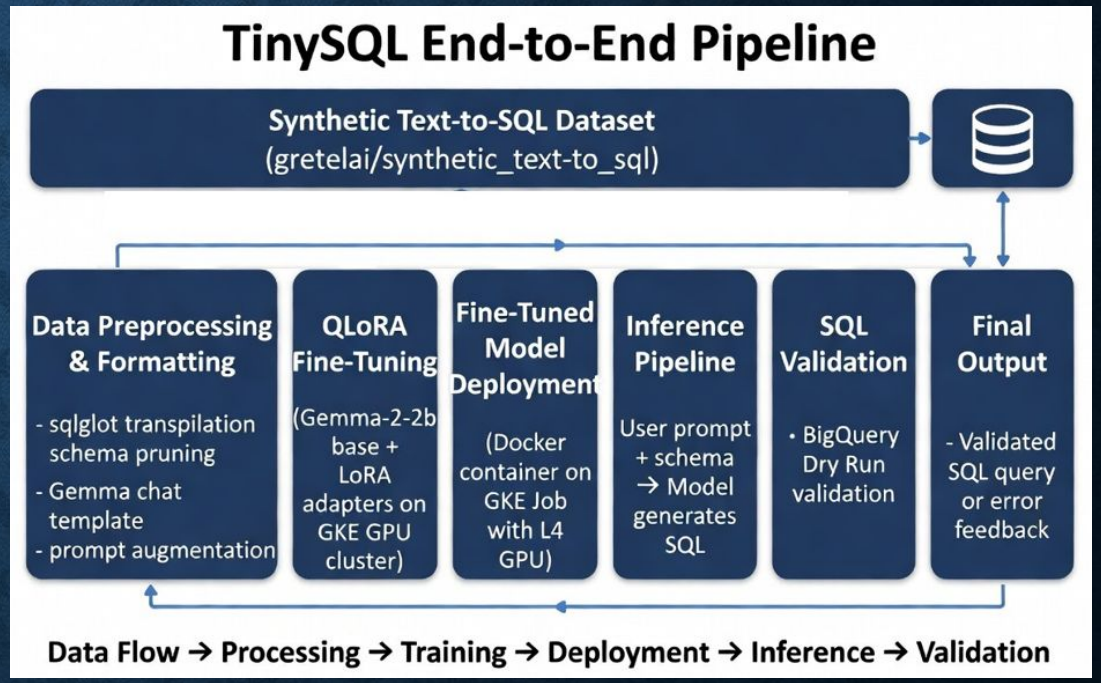
- Fine-tune **Google Gemma-2-2b** (and explore Gemma-3) for Text-to-SQL
- Ensure strict adherence to provided database schema
- Achieve high dry-run validation accuracy on BigQuery
- Build scalable training & inference pipeline on **GKE with GPU**
- Implement robust MySQL → BigQuery transpilation & cleaning
- Deliver comprehensive evaluation & visualizations

Dataset

- Dataset: gretelai/synthetic_text_to_sql
- ~100k+ training samples (question + schema + gold SQL + explanation)
- Rich variety: different complexities, task types, domains
- Preprocessing steps:
  - Dialect transpilation (MySQL → BigQuery)
  - Schema augmentation (distractors)
  - Sample row grounding
  - Prompt style variation
  - Gemma chat template formatting

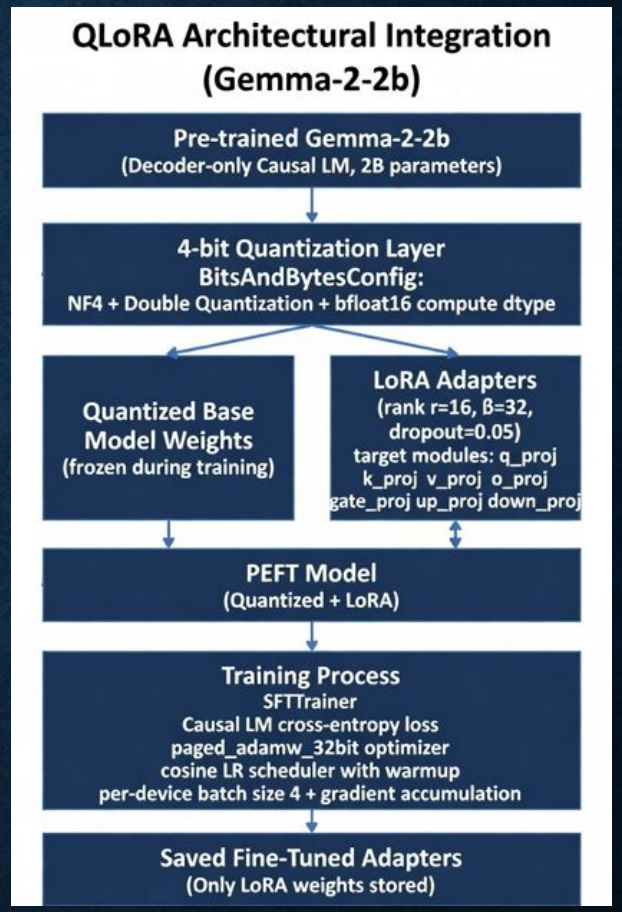
High-Level Architecture

- **Data Layer:** GCS Bucket + BigQuery Dataset
- **Compute Layer:** GKE Cluster (CPU + GPU node pool with NVIDIA L4)
- **Orchestration:** Kubernetes Jobs + GCS Fuse CSI Driver
- **Model Layer:** Gemma-2-2b (base) → QLoRA fine-tuned
- **Validation Layer:** BigQuery Dry-Run + sqlglot transpilation



Infrastructure Setup (GKE + GPU)

- Zonal GKE cluster (sdk-training-cluster)
- GPU Node Pool: g2-standard-16 + 1× NVIDIA L4
- Artifact Registry for custom Docker image (gemma_training)
- GCS Fuse CSI for direct bucket mounting (/gcs/{bucket})
- Workload Identity + Kubernetes Service Account
- Automated via create_gke_cluster.sh



Project Structure & Code Organization

- config/ → config.yaml (GCP, GKE, LoRA, hyperparams)
- job/ → Kubernetes Job template
- setup/ → Cluster & kubectl scripts
- src/util/ → config_loader.py, sql_utils.py (transpilation, validation)
- src/model_training/ → Benchmarking, SFT training, custom inference
- notebook/ → Main orchestration & analysis

SQL Utilities – The Secret Sauce

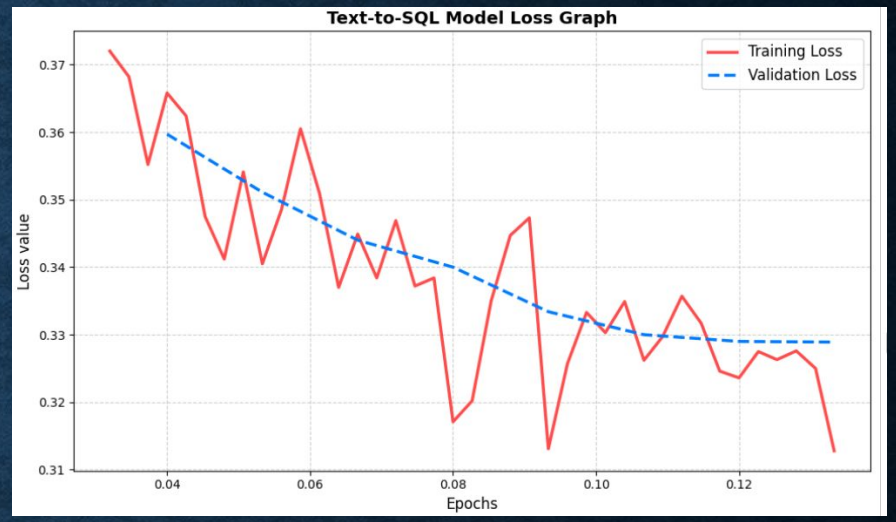
- sql_utils.py key functions:
  - extract_and_fix_ddl(): MySQL → BigQuery + strip constraints
  - transpile_to_bigquery()
  - prefix_table_names() → Forces {DATASET_ID}.table
  - validate_sql_dry_run(): BigQuery dry-run validation
- Handles hallucinations, dialect issues, and schema violations

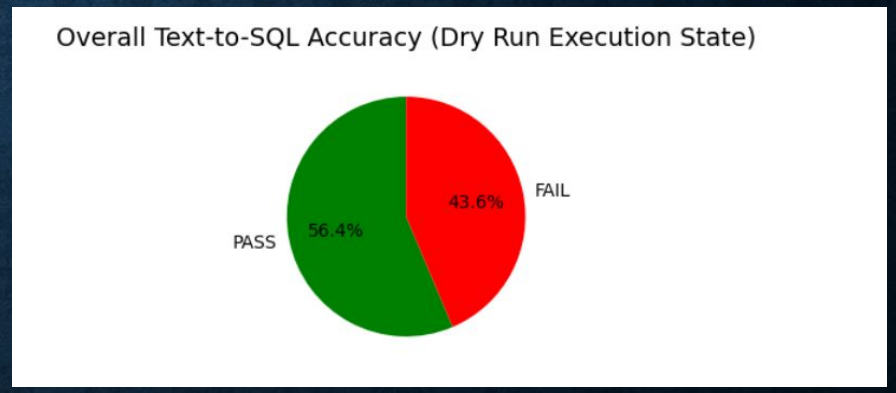
Prompt Engineering & Data Formatting

- Strict instruction prompt:
  - "You are a GoogleSQL expert"
  - "Use ONLY columns from schema"
  - "Prefix tables with dataset"
  - "Return ONLY the SQL query"
- Training data formatted to mirror exact inference template
- Uses <start_of_turn> (Gemma-2) or Gemma chat template

Model Training – QLoRA Fine-Tuning

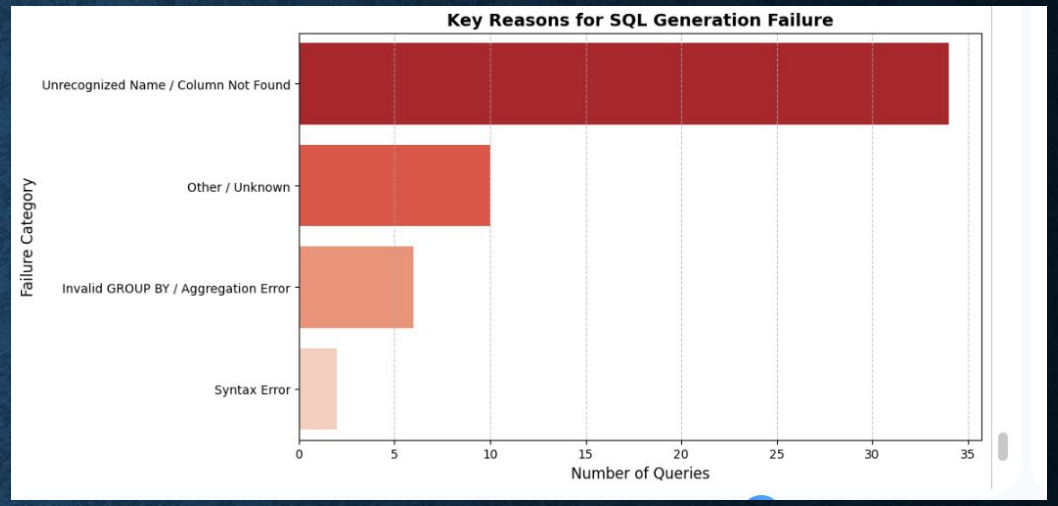
- Base Model: google/gemma-2-2b
- Technique: **QLoRA** (4-bit quantization) + LoRA on attention modules (q_proj, k_proj, etc.)
- Trainer: SFTTrainer from TRL
- Hyperparameters: batch_size=4, lr=2e-4, epochs=1–3, gradient accumulation
- Logging to GCP ML Diagnostics
- Output: LoRA adapters saved to GCS

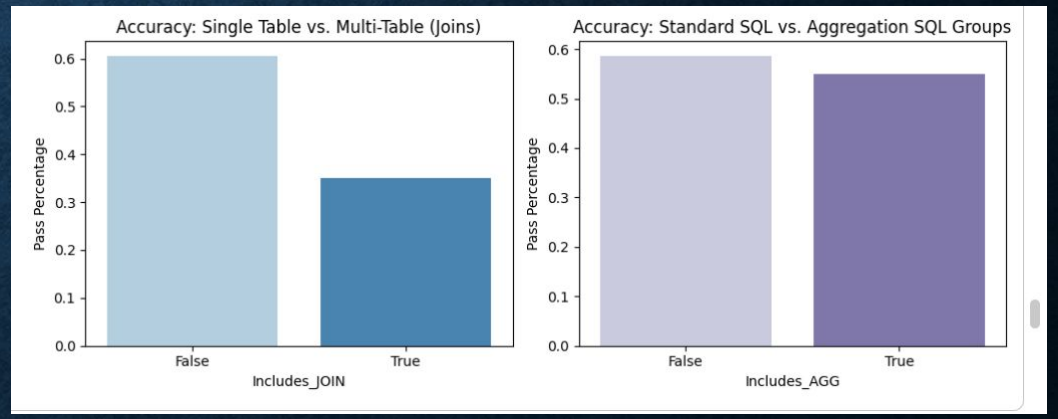




Inference & Benchmarking Pipeline

- Two modes:
  - Vanilla benchmarking (gemma_vanilla_benchmarking.py)
  - Fine-tuned inference (custom_inference.py)
- Process: Load record → Create table from DDL → Generate SQL → Clean → Dry-run validate → Log
- Runs as Kubernetes Job with GCS mount

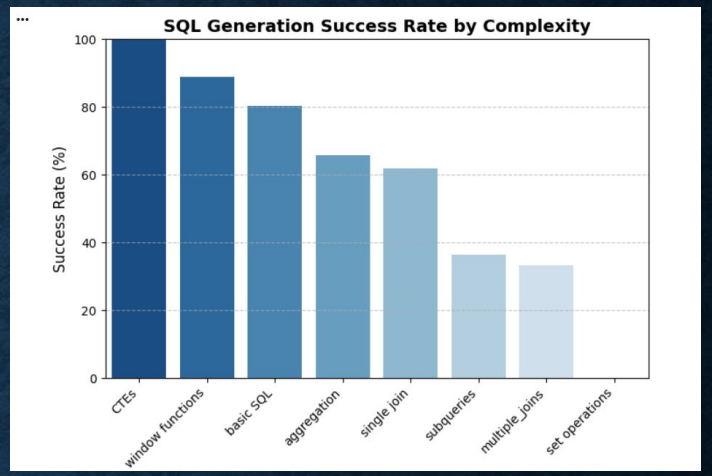


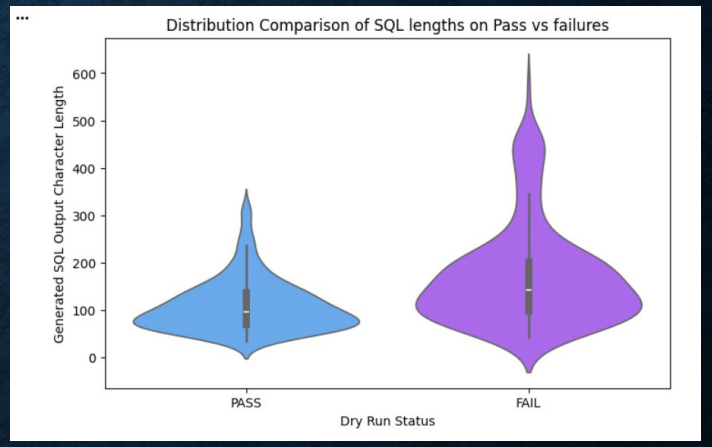


University of San Diego®

Results – Accuracy & Analysis

- Dry-run Pass Rate (Vanilla vs Fine-tuned)
- Success rate by SQL complexity
- Top failure categories (Unrecognized name, Syntax, Aggregation errors, etc.)
- Token accuracy during training (~91.5% on validation)
- Loss curves showing good generalization (no overfitting)

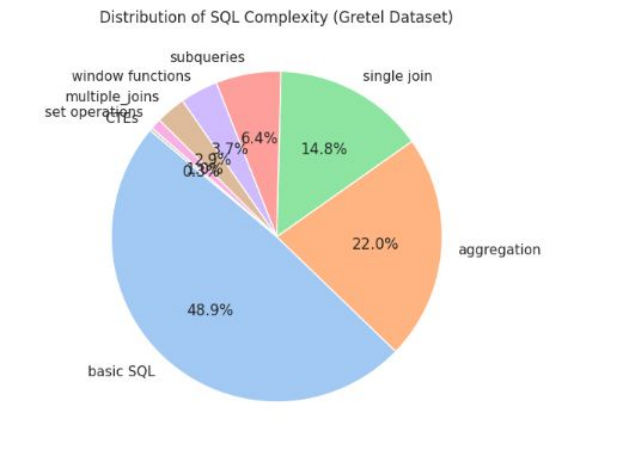


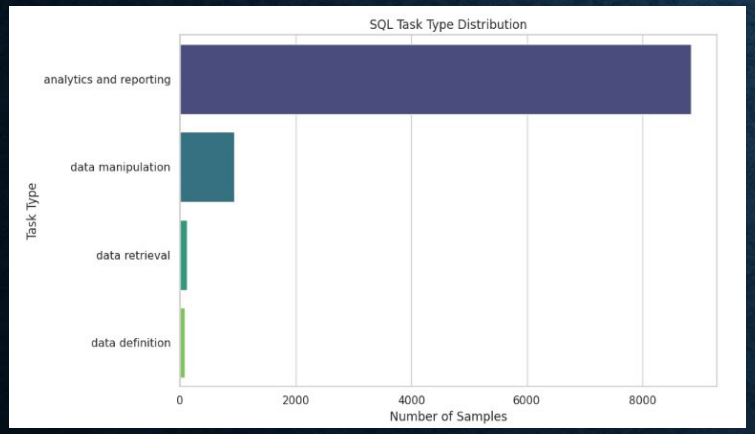


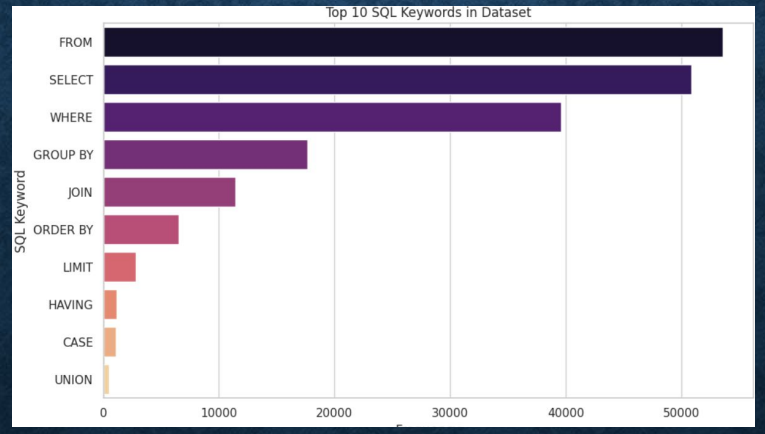
University of San Diego®

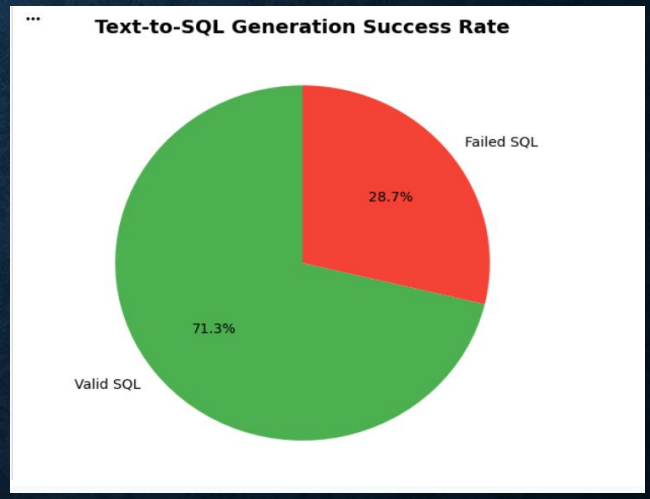
Visualizations from Notebook

- SQL Complexity & Task Type Distribution
- Top SQL Keywords & Frequently Joined Tables
- Prompt vs SQL Length Analysis
- Failure Category Breakdown









University of San Diego®

Challenges & Solutions

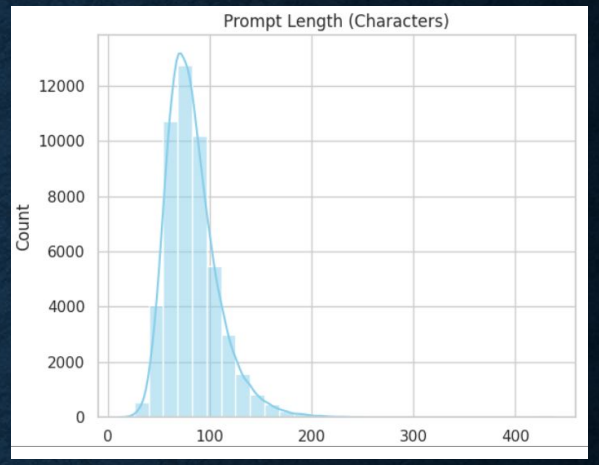
- Table

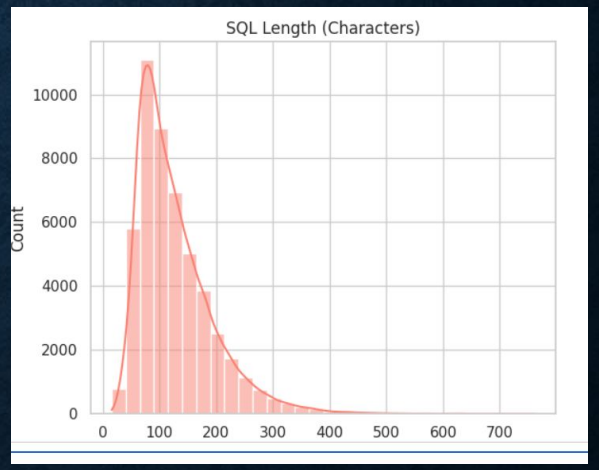
Challenge	Solution
Dialect differences	sqlglot transpilation + cleaning
Schema hallucination	Strict prompting + prefixing
Invalid SQL	BigQuery dry-run validation
Resource constraints	QLoRA + GKE GPU node pool
Large-scale execution	Kubernetes Jobs + GCS Fuse
Output cleaning	Custom extract_sql_only()

- Figure

Key Learnings & Best Practices

- Schema grounding + strict instructions are critical for Text-to-SQL
- Validation (dry-run) >> Execution for safe benchmarking
- QLoRA enables efficient fine-tuning on modest hardware
- Modular utilities (sql_utils) make the pipeline robust
- GKE + GCS Fuse = excellent MLOps foundation on GCP





Future Work & Improvements

- Experiment with larger Gemma models (Gemma-3 series)
- Add Retrieval-Augmented Generation (RAG) for very large schemas
- Implement self-consistency / multi-sample voting
- Agentic workflow (query repair on failure)
- Production API serving via Vertex AI or Cloud Run
- Expand to multi-turn SQL conversations

Conclusion

- Successfully built an end-to-end Text-to-SQL pipeline for BigQuery
- Demonstrated significant improvement through targeted fine-tuning + validation
- Delivered production-grade infrastructure on GKE
- Achieved reliable, schema-aware SQL generation

References & Tools

- Google Gemma models & Hugging Face
- TRL, PEFT, Transformers, BitsAndBytes
- sqlglot, Google Cloud (GKE, Artifact Registry, BigQuery, GCS)
- Gretel AI synthetic dataset
- AAI-590 Course Materials

References & Tools

- Live demo links or screenshots
- Kubernetes job logs
- Sample generated SQL (good vs improved)
- GitHub repo links:
  - https://github.com/santa-dask/aai-590-tinysql
  - https://github.com/sanjaykr33/aai-590-tinysql